HDOC DAY-57

Write a C program that reads the size and elements of a variable-length integer array, arr[]. Pass the array to a function named max_subarray_sum() using its address. Use pointers to pass the array between the calling function and max_subarray_sum(). The function max_subarray_sum() must:

- Find the maximum sum of any contiguous subarray in arr[] using Kadane's algorithm.
- Display the maximum subarray sum.
- If all elements are negative, display the largest, or least negative, element.
- Ensure that the selected subarray contains at least one element.

Example 1:

Array: -2 1 -3 4 -1 2 1 -5 4

Output:

Maximum subarray sum: 6

[Explanation: The contiguous subarray with the maximum sum is: 4 + (-1) + 2 + 1 = 6]

Example 2:

Array: -8 -3 -6 -2 -5 -4

Output:

Maximum subarray sum: -2

[Explanation: All elements are negative. Therefore, the largest element, -2, is selected.]

Example 3:

Array: 5 -2 3 4

Output:

Maximum subarray sum: 10

[Explanation: The entire array has the maximum sum: 5 + (-2) + 3 + 4 = 10]

Algorithm

1. Start.
2. Read the size n of the array.
3. Declare a variable-length integer array arr[n].
4. Read n elements into the array.
5. Pass the address of the array to max_subarray_sum().
6. Initialize current_sum and max_sum with the first array element.
7. For every remaining element:
    - Either start a new subarray from the current element, or add the element to the existing subarray.
    - Store whichever gives the larger sum in current_sum.
    - If current_sum > max_sum, update max_sum.
8. Return max_sum.
9. Display the maximum subarray sum.
10. Stop.

• C Program:

```
#include <stdio.h>


int max_subarray_sum(int *arr, int n)
{
  int current_sum = *arr;
  int max_sum = *arr;

  for (int i = 1; i < n; i++)
  {
    if (*(arr + i) > current_sum + *(arr + i))
      current_sum = *(arr + i);
```

```
        else
            current_sum = current_sum + *(arr + i);

        if (current_sum > max_sum)
            max_sum = current_sum;
    }

    return max_sum;
}

int main()
{
    int n;

    printf("Enter the size of array: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter %d elements: ", n);

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
```

```
    int result = max_subarray_sum(arr, n);

    printf("Maximum subarray sum: %d\n", result);

    return 0;
}
```

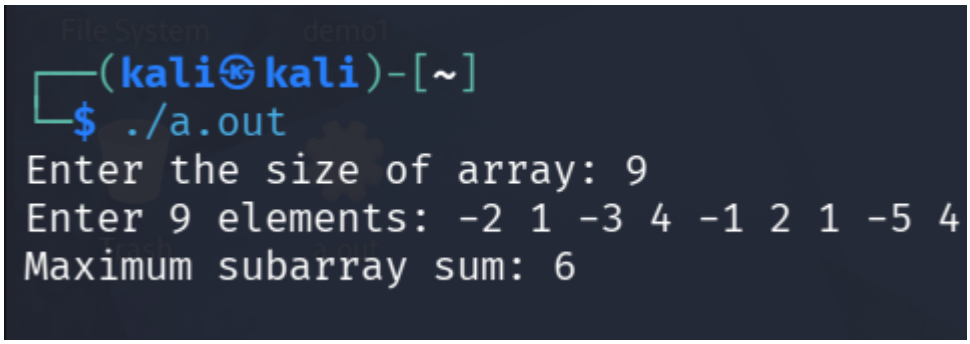
Test Case Table:

Test Case	Input array	Maximum Subarray	Expected Output
1	-2 1 -3 4 -1 2 1 -5 4	4 -1 2 1	6
2	4	-2	-2
3	-8 -3 -6 -2 -5 -4	5 -2 3 4	10
4	5 -2 3 4	1 2 3 4 5	15
5	-5 4 -1 7 -8	4 -1 7	10

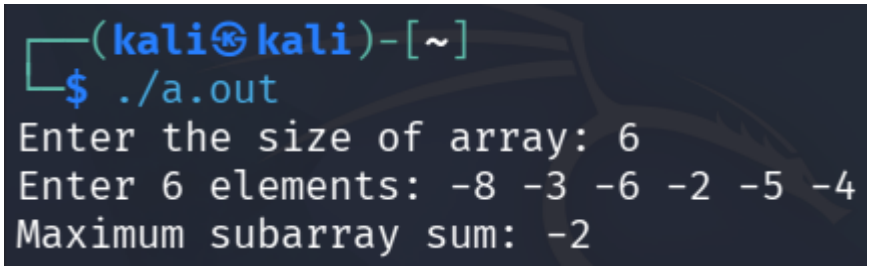
Compilation and Execution:

The following program has been compiled and executed successfully.

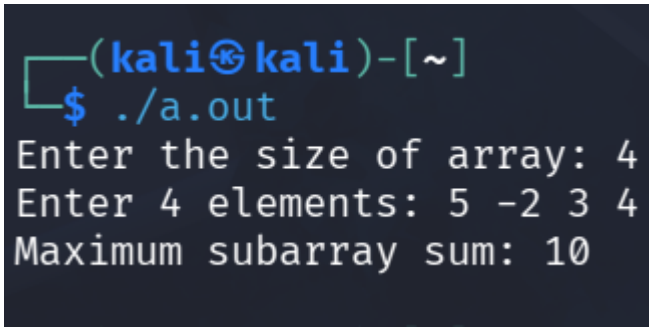
Test Case 1:



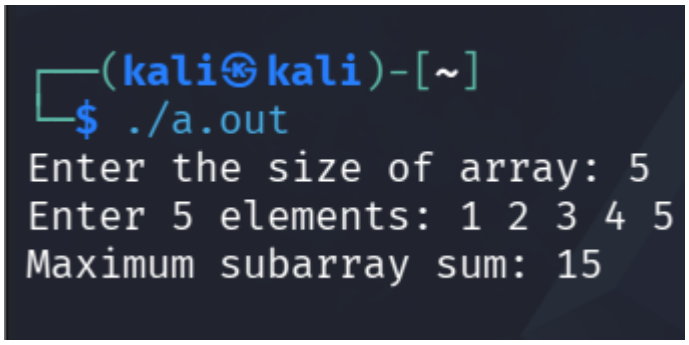
Test Case 2:



Test Case 3:



Test Case 4:



Test Case 5:

